What is a Data Structure?

A data structure is defined as a particular way of storing and organizing data in our devices to use the data efficiently and effectively. The main idea behind using data structures is to minimize the time and space complexities. An efficient data structure takes minimum memory space and requires minimum time to execute the data.

Lists:

A List acts as a Dynamic Array. It stores elements in contiguous memory blocks, allowing for instant mathematical indexing.

Dynamic Resizing:

Because they are contiguous, lists cannot grow infinitely in their current memory slot.
When a list reaches its capacity limit:
1 A new, larger contiguous block of memory is allocated.
2 All existing elements are copied to the new block (O(N) operation).
3 The old memory block is freed. This is why appending is O(1) amortized the O(N) resize happens rarely.

This is why appending is O(1) amortized—the O(N) resize happens rarely.

That's why adding a new element (appending) to a list is usually O(1), because resizing the list happens very rarely.

1. Appending (adding a new value)

- Most of the time, it is very fast
- Time complexity: O(1)

⚠ 2. When is it slow?

- When the list becomes full
- A new larger memory block is created
- All elements are copied → O(N)

🔻 Why is this not a big problem?

☞ Because this slow resizing operation happens very rarely

☞ So, appending is considered O(1) on average (amortized O(1)), because the expensive O(N) resizing happens only occasionally.

Problem: Given a list of unsorted integers, find the largest number without using built-in functions.

```
def find_max(numbers):
    maximum = numbers[0]
    for num in numbers:
        if num > maximum:
            maximum = num
    return maximum



# start here
arr = [10, 45, 2, 89, 33, 100, 67]
result = find_max(arr)
print("Maximum value is:", result)
```

Queue:

A Queue enforces the First-In, First-Out (FIFO) rule. Elements are added to the rear and removed from the front.

Working Principle:

- Elements are **inserted (enqueue)** at the **rear (end)**
- Elements are **removed (dequeue)** from the **front (beginning)**

☞ This means the element that is inserted first will be removed first.

Example:

If we insert elements: 10, 20, 30

- Insert → 10 → 20 → 30
- Remove → 10 (first inserted element)

Hash Function:
A **Hash Function** is a function that converts a key (e.g., a string) into an integer index, which is used to store and retrieve values in a hash table (dictionary).

In Python dictionary, when we insert a key-value pair, the hash function transforms the key into an array index.

◆ Example:

```
data = {}
data["Apple"] = 5
data["Banana"] = 8
val = data["Apple"]
```

◆ Explanation:

- `"Apple"` is passed through a hash function → produces an index (e.g., 3)
- Value `5` is stored at that index
- `"Banana"` is hashed → another index (e.g., 7)
- Value `8` is stored there
- When accessing `data["Apple"]`, the same hash function is applied, and value `5` is retrieved

🔹 Collision

A **Collision** occurs when two different keys produce the same hash index.

◆ Example:

- `"Apple"` → index 3
- `"Mango"` → index 3
  ☞ Both try to store in the same location

🔹 Collision Resolution

Modern dictionaries (like Python) resolve collisions using techniques such as **open addressing**.

- If a position is occupied, another empty position is searched
- This ensures that data can still be stored and accessed correctly

🔹 Performance

- Average case: **O(1)** (very fast)
- Worst case (many collisions): **O(n)**

নিচে **exam-ready structured answer** দেওয়া হলো 👇

🔹 Set (Python)

A set is an efficient data structure for storing **unique elements** without order, and it provides **fast lookup operations** using hash table implementation.

🔹 Key Characteristics:

1. Uniqueness

All elements in a set are **unique**.
Duplicate values are **automatically ignored**.

👉 Example:

```
s = {1, 2, 2, 3}
```

Result: `{1, 2, 3}`

2. Unordered

A set is **unordered**, meaning:

- Elements have **no fixed position or index**
- You cannot access elements using index like a list

3. Performance

Set operations are very efficient due to hashing:

- Checking existence (`x in set`) is **very fast**

👉 Average Time Complexity: **O(1)**